

[Experiment, Analysis, and Benchmark] EviStateBench: A Benchmark for Temporal Task-State View Maintenance over Embodied Observation Streams

Jiaxiang Zhou¹, Yizheng Huang¹, Yiqun Zhou¹, Ziming Zhao¹

¹Zhejiang University, Hangzhou, China

{jiaxiang, Yizhengz, yiqunz, zhaoziming}@zju.edu.cn

Abstract—Embodied agents produce observation streams from perception modules, action logs, trackers, task monitors, and simulators. These streams are often noisy, delayed, out of order, incomplete, and contradictory. Downstream task execution, however, needs queryable task-state views with temporal validity, information-availability semantics, uncertainty, and provenance: whether an object was inside a container at a valid time, whether that fact was known before late evidence arrived, which observations support or contradict it, and whether task goals currently hold. Existing embodied benchmarks mainly evaluate task completion, navigation, question answering, or memory recall; they do not isolate this data-management workload. We present EviStateBench, an Experiment, Analysis, and Benchmark contribution for temporal task-state view maintenance over embodied observation streams. EviStateBench constructs hidden state timelines from simulator truth and task specifications, releases sanitized StateObservation streams as public input, and evaluates standardized QueryAnswer predictions against hidden answer sets. Its workload covers bitemporal state, provenance, change sets, uncertainty sets, preconditions, failure localization, and goal-state queries. The reported artifact contains 570 simulator-grounded episodes from 36 activities, 5,848 hard queries, 2.86M clean observations, and 2.80M hard mixed observations; public-artifact, query-semantics, and answer-schema validation all pass. We also provide EviStateDB, a reference baseline that materializes bitemporal task-state views with support and contradict evidence. Experiments with recall-memory, temporal-log, and EviStateDB baselines show that EviStateBench separates retrieval, log-based lookup, and materialized temporal-state maintenance: exact accuracy ranges from 25.2% to 75.6%. The remaining errors concentrate in structured, set-valued, and task-derived queries, identifying open stress points for data systems over embodied evidence streams.

Index Terms—Benchmarking, Temporal Data Management, Bitemporal Data, Stream Processing, Incremental View Maintenance, Embodied AI

I. INTRODUCTION

Embodied agents increasingly observe the world through streams rather than through clean symbolic states. A robot may receive a detector output that a cabinet is open, a relation estimate that a cup is inside the cabinet, an action log that a place operation succeeded, and a later visual observation that contradicts the earlier relation. These records may come from different sources, arrive at different times, and have different

reliability. Some describe the current world state, while others arrive late and refer to an earlier event. Some evidence is missing because an object was occluded; some evidence is wrong because perception failed. A task executor nevertheless needs database-like answers: does the goal hold now, what would the system have known at a previous transaction time, which states changed during the episode, and which observations support or contradict an answer?

This is a data-management problem that is not isolated by current embodied benchmarks. BEHAVIOR introduced household activities with predicate-logic initial and goal conditions [1], while BEHAVIOR-1K scales this setting to long-horizon mobile manipulation in a richer simulation environment [2]. Habitat and Habitat 2.0 support embodied navigation and rearrangement evaluation [3], [4]. AI2-THOR provides an interactive 3D environment for visual AI [5], and VirtualHome represents household activities as executable programs [6]. TEACH studies embodied agents that interact through dialogue [7]. OpenEQA evaluates open-vocabulary embodied question answering over episodic memory and active exploration [8], and ALFRED evaluates grounded instruction following for everyday tasks [9]. These benchmarks expose important task, language, perception, navigation, and interaction challenges. Their primary scoring target, however, is not whether a data system can maintain a temporal task-state view from imperfect public observation streams with valid-time semantics, transaction-time availability, uncertainty, provenance, and structured query workloads.

The distinction matters for database systems. In a task such as placing a cup into a cabinet and closing the door, the relevant maintained states may include `inside(cup, cabinet)`, `open(cabinet)`, and `grasped(robot, cup)`. The query *was the cup inside the cabinet after the place action?* is not simply a nearest-frame retrieval query; it is a temporal state query over evidence. The query *what changed between the beginning and end of the episode?* is not a single symbolic-state lookup; it is a change-set query over a maintained view. The query *would the system have known this before the delayed observation arrived?* requires event time and arrival time to be separated,

a distinction central to temporal data management [10], [11]. Since observations arrive continuously and may be processed out of order, the workload also inherits concerns from stream processing [12]. Because task-state views must be updated after new evidence, corrections, and late arrivals, it is naturally related to incremental view maintenance [13], [14]. Finally, because evidence can be incomplete, noisy, or contradictory, the benchmark connects to uncertain data and provenance-aware data management [15]–[17].

We introduce EviStateBench, a data-management benchmark for temporal task-state view maintenance over embodied observation streams. A EviStateBench task instance provides public task specifications, StateObservation streams, queries, and metadata. It withholds hidden state timelines and hidden answer sets used by the evaluator. Systems under test may use any internal representation, including retrieval memory, temporal logs, SQL scans, rules, materialized views, learned memory, or hybrids. They are compared only through standardized QueryAnswer predictions under a shared evaluator. This public/hidden split is central to the benchmark: the oracle is derived from simulator truth and task specifications, not from any submitted system or reference implementation.

The reported EviStateBench artifact contains 570 simulator-grounded episodes over 36 activities, 5,848 hard queries, 2.86M clean observations, and 2.80M hard mixed observations. The workload covers bitemporal state, provenance, change sets, uncertainty sets, preconditions, failure localization, and task-goal queries. Public-artifact validation, query-semantics audit, and answer-schema validation pass without errors. Experiments with recall-memory, temporal-log, and materialized temporal-state baselines show that the benchmark separates retrieval-style memory, log-based temporal lookup, and explicit temporal view maintenance: exact accuracy ranges from 25.2% to 75.6%, while structured, set-valued, and task-derived queries remain difficult.

We also provide EviStateDB, a reference baseline engine for EviStateBench. EviStateDB maintains a TemporalStateView with valid-time intervals, transaction-time revisions, support and contradict evidence, confidence, and task-derived views. It is useful for diagnosing what an explicit temporal view-maintenance engine can do under the benchmark, but it is not the benchmark oracle. This separation lets EviStateBench evaluate EviStateDB alongside simpler or future systems under the same public inputs, hidden answers, and scoring protocol.

This paper makes four contributions.

- We formulate temporal task-state view maintenance over embodied observation streams as a data-management benchmark problem, separating it from perception, control, task completion, and memory-recall evaluation.
- We define the EviStateBench artifact boundary, StateObservation schema, query workloads, perturbation regimes, QueryAnswer schema, and evaluator semantics for public-input/hidden-oracle evaluation.
- We describe a simulator-grounded generation pipeline that produces hidden state timelines, clean and perturbed public observation streams, query sets, hidden answer

sets, validation reports, and reproducible benchmark manifests.

- We provide EviStateDB as a reference baseline engine and an experimental protocol for comparing retrieval, log-based, and materialized temporal-state maintenance strategies under shared public inputs and hidden oracle answers.

The intended EAB contribution is therefore not a new perception model, planner, or robotics controller. It is the benchmark object: a standardized artifact, a diagnostic query workload, a public/hidden evaluation boundary, an evaluator, and an analysis protocol for systems that maintain temporal task-state views. This follows the tradition of database benchmark papers such as LDBC for graph workloads [18], [19], TSM-Bench for time-series systems [20], and FinBench for financial graph transaction workloads [21]. It also aligns with recent EAB work that contributes evaluation frameworks and diagnostic analyses, including SQLMorph [22], BEACON [23], and plan-based AQP analysis [24].

II. MOTIVATION AND BENCHMARK SCOPE

A. A Running Task-State Example

Consider a household task in which an agent must put a cup into a cabinet and then close the cabinet. The task can be described by predicates over objects and time: the cup should become `inside(cup, cabinet)`, the cabinet should eventually be not `open(cabinet)`, and the robot should no longer be `grasped(robot, cup)`. A simulator or action script can provide the underlying state changes, but a deployed system observes only a stream of partial evidence. A relation detector may report `inside(cup, cabinet)` with moderate confidence after the place action. A visual detector may miss the cup once it becomes occluded. An action log may assert that the place operation succeeded. A later observation may arrive after the close action and revise an earlier belief. This example separates three objects that are often conflated. The *world state* is what actually holds at a valid time. The *observation evidence* is what a source claims, possibly late, incomplete, or incorrect. The *maintained task-state view* is what a system can answer after ingesting the public stream. EviStateBench evaluates the third object against hidden answers derived from the first object, while exposing only the second object as public input.

B. What the Benchmark Evaluates

EviStateBench evaluates systems that maintain queryable temporal views of task state. A benchmark instance provides a task specification, an observation stream, and a query set. A system outputs predicted answers, and the evaluator scores them against hidden answer sets. The system is not required to expose its internal state representation. It may use a log scan, a materialized view, relational tables, a retrieval index, rules, a learned memory module, or a hybrid architecture. The only required output is the standardized predicted QueryAnswer. The workload targets eight query families. `CHECK_STATE`

TABLE I

BENCHMARK POSITIONING. EXISTING EMBODIED BENCHMARKS EVALUATE TASK EXECUTION, SIMULATION, INSTRUCTION FOLLOWING, QUESTION ANSWERING, OR MEMORY RECALL. EVIStateBENCH ISOLATES TEMPORAL TASK-STATE VIEW MAINTENANCE FROM IMPERFECT OBSERVATION STREAMS.

Benchmark family	Primary target	Temporal state queries	Event/ arrival time	Perturbed observations	Task-derived views	Hidden oracle
BEHAVIOR / BEHAVIOR-1K	Household task execution and simulation	Partial	–	–	Partial	Simulator truth
ALFRED	Instruction following	–	–	–	Task success	Demo labels
OpenEQA	Embodied question answering	Episodic context	–	–	NL answers	Human/LLM scoring
Retrieval memory evals	Recall from stored observations	Limited	–	Usually no	–	Dataset labels
EviStateBench	Temporal task-state view maintenance	✓	✓	✓	✓	Simulator truth + specs

Note: ✓ indicates that the property is explicitly evaluated as a first-class scoring target; “Partial” indicates that the property may appear in task definitions or simulator state but is not the primary evaluation target.

and CHECK_GOAL are compact point probes over task predicates and goal conditions. AS_OF_STATE asks what would be known for a valid time at a specified transaction time, thereby testing information-availability semantics under late evidence. WHY_STATE asks for supporting or contradicting evidence. STATE_DIFF asks which states changed between two valid-time points or intervals. FIND_UNCERTAIN_STATES, CHECK_PRECONDITION, and FAILURE_LOCALIZATION expose set-valued uncertainty, task-precondition reasoning, and failure-analysis views.

C. What the Benchmark Deliberately Excludes

EviStateBench is not a robot-control benchmark. It does not score low-level manipulation success, exploration policy, path planning, visual perception quality, or language-grounded action execution as primary outcomes. These components are important, but including them in the main score would make failures difficult to attribute. An accuracy difference could reflect a detector, controller, simulator rollout, or planner rather than the temporal state-maintenance method being evaluated. The main artifact therefore uses simulator-grounded hidden timelines and controlled public observation streams. This choice is a scope restriction, not a claim that perception and control are unimportant. It makes the stream, queries, hidden answers, and evaluation semantics stable enough to evaluate temporal view maintenance reproducibly. Action-source and perception-source gaps are reported as limitations; the main correctness claim is over the controlled data-management workload.

D. Benchmark Requirements

The running example yields five benchmark requirements. (i) *R1: Temporal semantics*. The benchmark must ask about current and historical task states, not only final task success. (ii) *R2: Information availability*. It must separate valid time from arrival time, so that late evidence can repair a previous state without giving systems credit for knowing it before it arrived. (iii) *R3: Evidence sensitivity*. It must represent support and contradict observations, because task states can be uncertain or conflicting under imperfect evidence. (iv) *R4: Task-derived views*. It must evaluate predicates in the context of task goals, preconditions, changes, and failure explanations,

not only isolated object attributes. (v) *R5: Reproducibility*. It must provide public inputs, hidden answers, validation reports, and a shared evaluator so that systems are compared under the same workload and scoring semantics. Table I positions these requirements against related benchmark families. The point is not that prior benchmarks are deficient for their own goals. Rather, they target different layers of the embodied stack. EviStateBench fills the data-management layer between observation-producing embodied systems and task executors that need queryable, auditable state.

III. PROBLEM DEFINITION

A. Episodes and Hidden State Timelines

An episode e is an execution instance of a task specification. A task specification defines the relevant objects, task-state predicates, initial conditions, goal conditions, and task-level metadata. Let $\mathcal{P}$ be the set of task-state predicates, such as *open*, *inside*, *toggled_on*, *covered*, *cooked*, *temperature*, *frozen*, and *attached*. A state atom has the form $p(\mathbf{a}) = v$, where $p \in \mathcal{P}$, $\mathbf{a}$ is a tuple of object identifiers or values, and v is a Boolean, categorical, or numeric value. For each episode, the generator constructs a hidden state timeline

$$H_e = \langle h_1, \dots, h_m \rangle, \quad (1)$$

where each event h_i records a state atom, its value, and either a valid-time interval or a transition time derived from simulator truth and the task specification. Hidden timelines are not part of the public input. They are used only to derive hidden answer sets for evaluation.

B. Public Evidence Streams and Logical Views

The public input is a sequence of StateObservations. Each observation is an evidence record

$$o = (\text{obs_id}, \text{episode_id}, \text{event_time}, \text{arrival_time}, \text{source}, \text{kind}, \text{predicate}, \text{arguments}, \text{value}, \text{confidence}, \text{polarity}, \text{evidence_ref}, \text{metadata}). \quad (2)$$

The *event time* is the time in the episode to which the observation refers. The *arrival time* is the time at which the system receives the observation. The two may differ, which creates

TABLE II
QUERY WORKLOADS IN EVISTATEBENCH.

Query family	Main input	Evaluated capability
CHECK_STATE	Predicate, arguments, valid time	Point state lookup over task predicates
AS_OF_STATE	Predicate, valid time, transaction time	Bitemporal availability and repair
WHY_STATE	Predicate, time, evidence scope	Provenance and evidence support
STATE_DIFF	Scope, start time, end time	Change-set maintenance over an episode
CHECK_GOAL	Goal spec, valid time	Task-derived goal view correctness
FIND_UNCERTAIN_STAT	Episode and state scope	Set-valued uncertainty tracking
CHECK_PRECONDITION	Task state and precondition scope	Task-derived precondition reasoning
FAILURE_LOCALIZATION	Goal subset and state scope	Failed-goal state localization

the need for transaction-time semantics. The *polarity* indicates whether the observation supports, contradicts, or corrects a state claim. The *evidence reference* links the observation to an artifact item, such as a frame identifier, trajectory identifier, source record, or simulator-derived annotation. Observation streams can be clean or perturbed. Clean streams contain simulator-derived evidence with minimal injected noise. Perturbed streams introduce missing observations, temporal disorder, low-confidence evidence, and contradictions. The reported release includes a target-aware hard mixed stream that combines these issues and serves as the primary stress condition.

The event-time/arrival-time distinction is central. If an observation with *event_time* = 10 arrives at *arrival_time* = 40, then a system answering at transaction time 20 should not be credited for using it. The same system answering at transaction time 50 may use that observation to repair the maintained view. Thus, valid time describes the modeled world, while transaction time describes information availability. In embodied streams, this mismatch arises naturally from delayed model inference, batch perception, communication delays, and post-hoc task-monitor updates. A TemporalStateView is the logical object being evaluated. For an episode, it can be viewed as a maintained relation

$$V = (\text{state_id}, \text{episode_id}, \text{predicate}, \text{arguments}, \text{value}, \text{valid_start}, \text{valid_end}, \text{transaction_start}, \text{transaction_end}, \text{confidence}, \text{support}, \text{contradict}, \text{status}, \text{revision_history}). \quad (3)$$

Valid-time fields record when a state is believed to hold in the world. Transaction-time fields record when the system knew or stored that belief. The support and contradict sets record evidence used in the maintained answer. A system under test is not required to materialize V or expose it. The relation defines the benchmark semantics, not an implementation requirement.

C. Queries and Answer Schema

A query q is a public request over an episode, a predicate or task-goal specification, and one or more time parameters. The

TABLE III
QUERYANSWER FIELDS AND EVALUATOR ROLE.

Field	Meaning
query_id	Public query identifier used to join predictions with hidden answers
status	One of known, unknown, uncertain, conflict, or n/a
value	Boolean, categorical, numeric, or structured predicate value
change_set	Added, removed, or changed states for STATE_DIFF
confidence	Optional system confidence for calibration and error analysis
evidence	Optional support or contradict observation identifiers

main query families are shown in Table II. Hidden answer sets are generated from hidden timelines and task specifications. For bitemporal queries, the hidden answer depends on both valid time and transaction time.

A predicted answer follows the QueryAnswer schema in Table III. The schema is deliberately small: it contains the query identifier, an answer status, a normalized value or change set when applicable, an optional confidence score, and optional evidence identifiers. The status can be known, unknown, uncertain, conflict, or n/a. This lets the evaluator distinguish a wrong Boolean value from a system that correctly reports that the public stream does not justify a definite answer. In the reported artifact, clean answers are mostly known, while hard-mixed answers include unknown, uncertain, and conflict states.

D. Answer Normalization and Scoring Semantics

The evaluator normalizes hidden and predicted answers before scoring. Predicate names, object identifiers, and categorical values are canonicalized using public task metadata. Boolean values are mapped to a fixed true/false representation. Numeric values are compared with predicate-specific tolerances when such tolerances are defined; otherwise, exact normalized matching is used. Change sets are represented as sets of normalized state atoms with before/after values, so systems are not penalized for output ordering.

Correctness is not merely value equality. For CHECK_STATE, a prediction is exactly correct when both status and value match the hidden answer, or when both answers mark the state as not applicable. For AS_OF_STATE, the same rule is applied after enforcing the requested transaction time. This prevents hindsight leakage: a system may know the final world state but still be wrong about what information was available at an earlier transaction time. For STATE_DIFF, exact match requires the full change set to match, while diff F1 gives partial credit for overlapping changed atoms. For CHECK_GOAL, exact match checks the task-level status and goal truth value, while goal-predicate F1 measures predicate-level agreement. Status semantics are scored explicitly because they are central to imperfect streams. A system should not be forced to guess a Boolean value when public evidence is insufficient. Conversely, it should not be rewarded for answering unknown on every difficult query. The evaluator therefore reports coverage and status accuracy alongside value metrics. This exposes the trade-off between

overconfident value prediction and conservative uncertainty reporting. Evidence fields are optional in the core score, but they are retained for audit and why-style workloads. When a system emits support or contradict observation identifiers, the evaluator can measure overlap with oracle evidence. The reported main score does not require evidence citation so that simple baselines remain comparable, but evidence references remain first-class fields in the public stream and answer schema because provenance is part of the benchmark design.

IV. EVISTATEBENCH BENCHMARK DESIGN

A. Benchmark Package and Artifact Boundary

EviStateBench is a benchmark package with three components: public artifacts, hidden evaluator artifacts, and a shared evaluation protocol. The public artifacts contain task specifications, StateObservation streams, query sets, metadata, manifests, validation reports, and build commands. The hidden evaluator artifacts contain hidden state timelines and hidden answer sets. Systems under test read only the public package and submit QueryAnswer predictions. The evaluator reads the predictions and hidden answers and computes the reported metrics. This boundary is central to the benchmark design. The public artifact exposes information that a deployed state-maintenance system could plausibly observe: task descriptions, observation records, query specifications, and metadata needed for parsing and normalization. It does not expose hidden timeline events, answer sets, or perturbation labels that reveal which observations were injected, removed, delayed, or contradicted. Thus, systems are evaluated on temporal task-state maintenance from public evidence, not on access to simulator truth. The same boundary also makes the benchmark representation-agnostic. A system may implement a retrieval memory, temporal log, SQL scan, rule engine, materialized view, learned memory module, or hybrid architecture. It is not required to expose its internal representation. All systems are compared through the same QueryAnswer schema and the same hidden-oracle evaluator.

EviStateDB is evaluated by EviStateBench; it does not generate the ground truth used to judge other systems. The oracle is constructed from simulator truth and task specifications through the benchmark generation pipeline. This separation avoids making the reference engine part of the labeling process and allows EviStateDB to be compared with simpler or future systems under the same public inputs and hidden answers. The package supports three independent roles. A *benchmark builder* regenerates public and hidden artifacts from pipeline manifests. A *system developer* downloads the public package, implements an answerer, validates prediction format, and emits predictions. An *evaluator* scores predictions against hidden answers. This separation is important for artifact review and public leaderboards: system developers can test parsers, run baselines, and validate output schemas without access to hidden timelines or answer files. The public-artifact validation step checks that required manifests exist, query identifiers are well formed, observation streams are readable, task specifications align with query metadata, and answer leakage is absent

from public files. In the reported artifact, validation covers 570 task specifications, 5,848 query rows, one primary hard mixed public stream, and reports 0 validation errors.

B. Workload Axes

EviStateBench varies several workload axes simultaneously so that results can be interpreted diagnostically rather than only as a single aggregate score. The main axes are query family, predicate family, episode horizon, stream regime, temporal position, and evidence status. The *query-family* axis determines the answer shape: point values, bitemporal state, provenance evidence, change sets, uncertainty sets, preconditions, failure localization, or goal-state answers. The *predicate-family* axis determines whether the maintained state involves unary object states, binary relations, numeric values, material states, or contact configurations. The *episode-horizon* axis controls stream length and the number of opportunities for stale evidence, late repair, and accumulated uncertainty. The *stream-regime* axis distinguishes clean streams from perturbed streams. The *temporal-position* axis distinguishes initial, transition-near, and final queries. The *evidence-status* axis captures whether the public stream provides support evidence, contradict evidence, both, or neither. These axes determine how results should be read. A system that performs well on clean final-state checks may still fail on transition-near as-of queries. A system that handles Boolean unary predicates may not handle numeric temperature states or relation changes. A system that answers short-episode goal queries correctly may degrade on longer episodes because stale evidence accumulates. EviStateBench therefore reports overall metrics together with query-family and stream-regime breakdowns.

C. Perturbation Regimes

The perturbation regimes are designed to stress data-management behavior rather than to model one specific sensor. Clean streams test basic state-view construction. Missing-observation regimes test unknown-state and stale-state handling. Temporal-disorder regimes test valid-time and transaction-time repair. Low-confidence regimes test confidence propagation and calibration. Conflict regimes test support/contradict evidence handling and status semantics. The reported hard mixed stream combines several perturbation types and serves as the primary stress condition. This choice reflects the benchmark target: embodied observation streams rarely contain a single isolated defect. A system must often handle missing evidence, delayed evidence, low-confidence evidence, and contradictory evidence in the same episode. The perturbations are target-aware in the sense that they stress evidence near task-relevant states, while preserving enough background observations for realistic stream structure.

Perturbation provenance is kept hidden from systems under test. Public streams contain the resulting observations, but they do not reveal which records were removed, downgraded, delayed, or injected. Hidden perturbation metadata is retained only for diagnostic analysis and validation. This prevents the

benchmark from becoming a label-reading task while still allowing error analysis by perturbation type.

D. Evaluation Modes

EviStateBench supports both online and batch evaluation modes. In online mode, a system ingests observations in arrival-time order and answers queries at their declared transaction times. This mode best matches task monitors that operate during execution and emphasizes update cost, state repair, and incremental view maintenance. In batch mode, a system may preprocess the full public stream before answering queries. However, it must still respect transaction-time constraints for bitemporal queries such as `AS_OF_STATE`. Batch mode is useful for comparing database implementations, retrieval systems, and offline analysis pipelines whose natural interface is not an online monitor. Both modes are valid, but they answer different performance questions. Online mode evaluates whether a system can maintain and repair task-state views incrementally. Batch mode evaluates whether a system can implement the benchmark semantics correctly over the public stream. When both modes are available for a system, EviStateBench reports preprocessing time separately from update time and query time, so that an offline full-stream scan is not confused with an online maintained view.

E. Evaluation Metrics

EviStateBench reports diagnostic metrics rather than a single leaderboard number. Exact accuracy checks whether the predicted structured answer matches the hidden answer for a query. Value accuracy evaluates the value field when applicable. Status accuracy checks whether the system distinguishes known, unknown, uncertain, conflict, and n/a. Coverage reports how often the system returns a valid answer in the required schema. Structured workloads use additional metrics. `STATE_DIFF` uses both exact change-set match and mean F1 over changed state atoms. `CHECK_GOAL` reports task-level correctness and goal-predicate F1, so that a system can receive partial credit when it identifies some but not all satisfied or violated goal predicates. Provenance-oriented workloads can score overlap between emitted support or contradict evidence and oracle evidence. For systems that emit confidence, EviStateBench also reports confidence mean absolute error where hidden confidence is defined. These metrics are intentionally diagnostic. A single global score would hide differences between stale-state errors, transaction-time errors, status-calibration errors, missing provenance, and change-set errors. EviStateBench therefore reports overall metrics together with query-family, stream-regime, and structured-answer breakdowns. This design lets the benchmark identify not only which system performs better, but also which data-management capability is responsible for the difference.

V. BENCHMARK GENERATION AND ARTIFACTS

A. Generation Pipeline

The generation pipeline starts from task specifications and simulator-grounded episode sources. It extracts semantic state

TABLE IV
REPORTED EVIStateBENCH ARTIFACT PROFILE.

Item	Count or status
<i>Scale</i>	
Episodes	570
Activities	36
Task families	4
Target predicates	11
Task specifications	570
Clean observations	2,856,815
Hard mixed observations	2,797,127
Queries	5,848
Primary public stream	hard_mixed
<i>Validation</i>	
Validation errors	0
Public validation status	PASS
Query semantics audit	PASS
Answer schema validation	PASS

changes from simulator truth, normalizes them into hidden timeline events, generates clean `StateObservation` streams, injects perturbations to produce stress streams, constructs query sets, derives hidden answer sets, and builds a sanitized public package. Each stage writes a manifest or validation report so that the benchmark can be rebuilt, audited, and compared across releases.

The hidden timeline is the anchor of the artifact. It records task-state transitions from which both public observations and hidden answers are derived. Clean observations are public evidence records generated from the same timeline. Perturbed observations preserve task structure while introducing stream defects that a maintenance system must handle. Query generation then samples state checks, bitemporal as-of probes, provenance queries, full-episode diffs, uncertainty-set queries, precondition checks, failure-localization queries, and task-goal checks. This pipeline enforces the public/hidden boundary. Hidden timelines, answer sets, and perturbation provenance are retained for evaluation and analysis, but they are not included in the public package. Systems under test receive only task specifications, public observation streams, query specifications, metadata, manifests, and validation files.

B. Query Generation and Selection

The query generator is tied to hidden timeline events and task goal specifications. It first constructs a candidate pool from task-relevant state changes, target predicates, and goal predicates. Candidate probes include `CHECK_STATE` queries at target valid times, paired `AS_OF_STATE` queries with transaction times before and after the relevant observation window, `WHY_STATE` provenance probes, episode-level `STATE_DIFF` queries, and initial/final `CHECK_GOAL` queries. The paired transaction-time probes test whether systems respect information availability rather than answering historical queries from final hindsight.

The reported hard split filters this candidate pool into the public workload shown in Table V. It keeps the temporal and provenance-heavy probes, keeps one

episode-level `STATE_DIFF`, `FIND_UNCERTAIN_STATES`, `CHECK_PRECONDITION`, and `FAILURE_LOCALIZATION` query per episode, and down-samples easier point-state and goal probes to 120 `CHECK_STATE` and 120 `CHECK_GOAL` queries. This two-stage design keeps the final public query set smaller than the full candidate pool while preserving temporal, set-valued, task-derived, and failure-analysis workloads.

The workload is therefore task-linked rather than randomly sampled over arbitrary predicate/time combinations. Queries target states that matter to task progress: object states, relation changes, material states, transitions, preconditions, and goals. For analysis, the generator records hidden links from queries to the timeline events or goal predicates that motivated them. These links are not exposed to systems under test, but they let the evaluator group errors by transition type, predicate family, goal predicate, or perturbation source.

C. Perturbation Injection

Perturbations are injected after clean streams are generated. Missing-observation perturbations remove evidence near selected state transitions. Temporal-disorder perturbations shift arrival times or reorder observations while preserving event-time references. Low-confidence perturbations lower confidence without necessarily changing the observed value. Conflict perturbations introduce observations whose polarity or value contradicts the hidden state. The hard mixed stream combines these effects and serves as the primary stress stream in the reported artifact.

The perturbation design is not meant to mimic every possible sensor failure. Its purpose is to create controlled failure modes that correspond to data-management challenges. Missing evidence tests whether a system can represent unknown or stale states. Temporal disorder tests bitemporal repair. Low confidence tests confidence propagation and calibration. Conflict tests evidence handling and status semantics. Mixed streams test whether systems remain robust when these issues co-occur. Perturbations are applied with deterministic seeds and written to hidden manifests. The public package exposes the resulting observation stream but does not reveal which observations were removed, downgraded, delayed, or injected. This design supports two levels of reporting. Main results compare public stream regimes such as clean versus hard mixed. Diagnostic analysis can additionally attribute failures to missing evidence, temporal disorder, low confidence, or contradiction without turning the public task into a perturbation-label reading problem.

D. Current Main Artifact

Table IV summarizes the artifact used in this paper, `public_final_570_hard_v1`. It contains 570 simulator-grounded episodes across 36 BEHAVIOR/OmniGibson activities and 5,848 hard queries. The clean stream contains 2,856,815 observations, and the hard mixed stream contains 2,797,127 observations. Public-artifact validation, query-semantics audit, and answer-schema validation all report PASS.

TABLE V
REPORTED QUERY DISTRIBUTION IN THE MAIN ARTIFACT.

Query family	Count	Share
AS_OF_STATE	1,772	30.3%
WHY_STATE	1,556	26.6%
STATE_DIFF	570	9.7%
FIND_UNCERTAIN_STATES	570	9.7%
CHECK_PRECONDITION	570	9.7%
FAILURE_LOCALIZATION	570	9.7%
CHECK_STATE	120	2.1%
CHECK_GOAL	120	2.1%
Total	5,848	100.0%

These counts characterize the release as a controlled simulator-grounded benchmark for temporal task-state maintenance, not as a claim of full real-world robot deployment coverage.

The query distribution emphasizes temporal and provenance-heavy state maintenance. `AS_OF_STATE` and `WHY_STATE` form the two largest query families, followed by one episode-level structured query from each of `STATE_DIFF`, `FIND_UNCERTAIN_STATES`, `CHECK_PRECONDITION`, and `FAILURE_LOCALIZATION`. The artifact also includes smaller samples of `CHECK_STATE` and `CHECK_GOAL` probes. The target predicates cover object unary state, material state, containment, and contact configuration, including `open`, `covered`, `inside`, `cooked`, `toggled_on`, `temperature`, `attached`, `frozen`, `max_temperature`, `hot`, and `on_fire`.

E. Validation, Scope Limits, and Reproducibility

The public package is validated before evaluation. Release validation verifies the task specifications, query rows, public observation streams, manifests, schema conformance, and absence of answer leakage in public files. The reported release validates 570 task specifications, 5,848 query rows, and the primary hard mixed stream with 0 validation errors. The artifact also records horizon splits for interpretation: 540 short controlled episodes and 30 long state-chain episodes.

Two scope limits are deliberately kept visible. First, the main artifact isolates temporal view maintenance from low-level robot control. Second, it uses structured simulator-derived observations rather than raw perception outputs as its correctness foundation. These choices make the benchmark reproducible and diagnostically clean: failures can be attributed to temporal state maintenance rather than to detector quality, controller robustness, simulator rollout, or task policy. Low-level rollout and perception-derived streams are outside the main score; the correctness claim is state-view maintenance given public observation streams.

The artifact identifier encodes the release lineage and hard-split role. The reported artifact, `public_final_570_hard_v1`, is therefore not an anonymous directory name, but a compact description of the benchmark package used by the reported experiments. Manifests record task specifications, stream files, query sets,

answer sets, validation reports, and profile summaries. A reproduction run verifies that these manifests agree with the paper tables before running baselines.

The public release includes manifests, validation scripts, baseline runners, predictions, and evaluator outputs for reproducing the reported results. It is available as version v1.0 at <https://github.com/TheShadowFliesAway/EviStateBench-ICDE2027-Artifact>.

VI. REFERENCE ENGINE AND EXPERIMENTAL PROTOCOL

A. EviStateDB as a Reference Baseline

EviStateDB is a reference baseline engine for EviStateBench. It ingests StateObservation streams and maintains a TemporalStateView over task-state atoms. For each atom, it records valid-time intervals, transaction-time revisions, confidence, support evidence, contradict evidence, status, and revision history. It also derives task-facing views required by the workload, including goal views, precondition views, state-diff views, uncertain-state views, and failure-localization views. EviStateDB is a baseline, not the benchmark oracle. The benchmark is designed to evaluate any system that can read the public artifacts and emit QueryAnswer predictions. EviStateDB provides one interpretable data-management implementation of bitemporal repair, evidence tracking, and task-derived view maintenance, but it does not generate the hidden answers used to judge other systems. Internally, EviStateDB stores an observation log and materializes state intervals indexed by episode, predicate, arguments, valid time, and transaction time. Task-derived views are computed from the maintained intervals rather than from hidden timelines or answer files. This design makes EviStateDB a concrete reference point for systems that trade update cost for query speed, while keeping EviStateBench representation-agnostic.

B. Maintenance Procedure

Algorithm 1 sketches the reference ingestion procedure used by EviStateDB. The procedure processes each StateObservation as an evidence update rather than as a direct overwrite of the current state. It first locates the state atoms and active valid-time intervals to which the observation may apply. It then updates the maintained view according to the observation polarity: support evidence strengthens an existing interval, contradict evidence revises confidence or status and may close or split an interval, and correction evidence replaces or repairs a previous claim. When an observation refers to an earlier event time but arrives later, EviStateDB records a transaction-time revision so that later queries can distinguish what the system knew before and after the late evidence arrived. The pseudocode is not a required implementation for systems under test. A submitted system may use SQL scans, log replay, materialized views, retrieval indexes, rules, learned memory, or hybrids. The purpose of Algorithm 1 is to make explicit the data-management operations that EviStateBench stresses: matching observations to state atoms, maintaining valid-time intervals, preserving transaction-time history, tracking support and contradict evidence, and refreshing task-derived views

Algorithm 1 Reference ingestion procedure in EviStateDB.

Require: StateObservation o , maintained view V

Ensure: Updated maintained view V

```

1:  $A \leftarrow$  state atoms matching  $o.predicate$  and  $o.arguments$ 
2:  $I \leftarrow$  active intervals for  $A$  at  $o.event\_time$ 
3: if  $o.polarity$  supports an interval in  $I$  then
4:   Add  $o$  to support evidence of the matched interval
5:   Update interval confidence and status
6: else if  $o.polarity$  contradicts an interval in  $I$  then
7:   Add  $o$  to contradict evidence of the matched interval
8:   Revise confidence and status
9:   Close, split, or mark the interval if required by the conflict
10: else if  $o.polarity$  corrects a previous claim then
11:   Apply the correction to affected valid-time intervals
12:   Record replaced evidence in the revision history
13: else
14:   Insert a new interval for the observed state atom starting at
        $o.event\_time$ 
15:   Initialize confidence, status, and support evidence from  $o$ 
16: end if
17: if  $o.arrival\_time > o.event\_time$  then
18:   Append a transaction-time revision for the affected state atoms
19: end if
20: Refresh indexes on episode, predicate, arguments, valid time, and transaction time
21: Refresh affected task-derived views
22: return  $V$ 

```

used by goal, precondition, state-diff, uncertainty, and failure-localization queries. Thus, EviStateDB serves as an interpretable reference point for the benchmark without constraining the representation choices of future systems.

C. Baselines and Fair-Comparison Rules

The reported evaluation compares EviStateDB with two transparent baselines. *Recall Memory* retrieves related observations and answers from retrieved evidence without maintaining an explicit temporal task-state view. *Temporal Log* stores the observation log and evaluates temporal queries over the log, but does not materialize the full set of task-derived views maintained by EviStateDB. These baselines are intentionally auditable: their role is to test whether EviStateBench distinguishes retrieval-style memory, log-based temporal lookup, and explicit temporal state-view maintenance.

All systems are evaluated under the same public inputs: task specifications, observation streams, query sets, and metadata. They must not read hidden timelines, hidden answer sets, or private perturbation labels. They may use external libraries, database engines, indexes, learned components, or rule systems, but all predictions must be emitted in the same QueryAnswer format. A single evaluator computes exact accuracy, value accuracy, status accuracy, coverage, structured-answer metrics, and efficiency metrics. This shared protocol is what turns EviStateBench from a data collection into an EAB benchmark. The stream, queries, hidden answers, and metrics are fixed; the system implementation is the only moving part. As a result, differences in reported performance can be attributed to the state-maintenance strategy rather than to different inputs, different labels, or different scoring code.

D. Reporting Protocol

Each run report records the public artifact identifier, stream name, system name and version, execution mode, hardware, random seed when applicable, preprocessing time, update time, query time, and output coverage. Accuracy and efficiency are reported separately. A system that answers accurately by scanning the full public stream for every query is different from a system that maintains a materialized view online; both are valid under EviStateBench, but they expose different accuracy-efficiency trade-offs. The reported runs use the same reporting template for all baselines. This is important for benchmark reproducibility: a small change in stream order, query filtering, hidden-answer version, or schema validation can invalidate comparisons even when each individual system runs successfully. EviStateBench therefore fixes the public artifact, hidden oracle, prediction schema, evaluator version, and metric definitions for every reported run.

VII. EVALUATION AND ANALYSIS

A. Artifact Characterization

The reported artifact supports the first EAB claim: EviStateBench is a concrete, validated benchmark package rather than only a proposal. Fig. 1 summarizes the released workload. The artifact contains 570 simulator-grounded episodes, 36 activities, four task-state families, 11 target predicates, 5,848 hard queries, 2.86M clean observations, and 2.80M hard mixed observations. The query set covers point state checks, bitemporal as-of probes, provenance queries, full-episode diffs, uncertainty sets, preconditions, failure localization, and task-goal checks. The target predicates cover object unary state, material state, containment, and contact configuration. This diversity is important because a benchmark limited to one predicate family would be easy to overfit with a specialized rule.

B. Hard-Split Difficulty

Fig. 2 explains why the hard split is more than a larger file. Among 14,905 target simulator-state observations, only 2,512 remain as direct target evidence, 10,282 are downgraded to lower-confidence evidence, and 2,111 are dropped. The overall stream also contains 87,571 dropped observations, 144,011 low-confidence observations, and 27,883 injected conflicts. This perturbation is target-aware: it stresses evidence near the states that queries ask about, while leaving most background observations intact.

The hard split is therefore a controlled stress test for data-management behavior. It is not an empirical model of every real sensor failure. Its purpose is to create missing, low-confidence, delayed, and conflicting evidence regimes in which systems must decide whether a state is known, unknown, uncertain, or conflicting. The confidence and predicate panels in Fig. 2 show that the resulting workload is not binary. Different predicates exhibit different answer mixtures, and temporal/provenance queries include high-confidence, low-confidence, missing, and conflicting cases.

C. Long State Chains

Most episodes in the reported artifact are short controlled transitions, but the release also includes a 30-episode long state-chain subset over cooking and freezing activities. The cooking chains move from an actuator state to temperature and maximum-temperature changes, and then to hot and cooked states. The freezing chain moves from opening the refrigerator to temperature change and then to frozen state. This subset is not large enough to claim broad long-horizon BEHAVIOR coverage, but it demonstrates that the benchmark can express multi-stage temporal state maintenance beyond a single target flip.

Fig. 3 makes this diagnostic role explicit. The long subset is small and is not a separate robotics claim; it shows that the artifact can express chained state consequences and convert them into temporal availability cases under hard mixed evidence.

D. Baseline Results

Fig. 4 compares three reference baselines on the final hard split. Recall memory answers from retrieved or recent evidence without maintaining a temporal state view. Temporal log keeps an explicit event log and evaluates temporal queries over it. EviStateDB maintains a bitemporal task-state view with support and contradict evidence.

The overall exact accuracies are 25.2% for Recall memory, 69.0% for Temporal log, and 75.6% for EviStateDB. These results support the benchmark’s diagnostic goal: the workload separates retrieval-style memory, append-only temporal lookup, and materialized temporal-state views. The largest gap is between retrieval-style memory and the two temporal baselines; the additional gain from Temporal log to EviStateDB is smaller and concentrates in structured and task-derived workloads. Temporal log and EviStateDB reach 98% exact accuracy on AS_OF_STATE and WHY_STATE, but they still struggle on structured workloads. EviStateDB reaches only 39% exact accuracy on STATE_DIFF and 10% on FIND_UNCERTAIN_STATES, although its partial structured metrics are higher: 42.4% Diff F1, 63.8% Goal F1, 72.6% State-set F1, and 92.5% Why-support F1.

The runtime panel reports per-query prediction time, not full end-to-end indexing or stream-loading time. It shows a different stress point: most point-state queries are inexpensive after indexing, while FIND_UNCERTAIN_STATES dominates p95 query time for all methods. This is useful benchmark information because it separates semantic difficulty from query-time pressure.

E. Failure and Difficulty Diagnostics

The baseline diagnostics separate two different messages. First, answer status matters. Recall memory fails especially on unknown, uncertain, and conflict answers, while Temporal log and EviStateDB recover most unknown and uncertain statuses. Second, the high predicate-level accuracy for Temporal log and EviStateDB on AS_OF_STATE and WHY_STATE should not

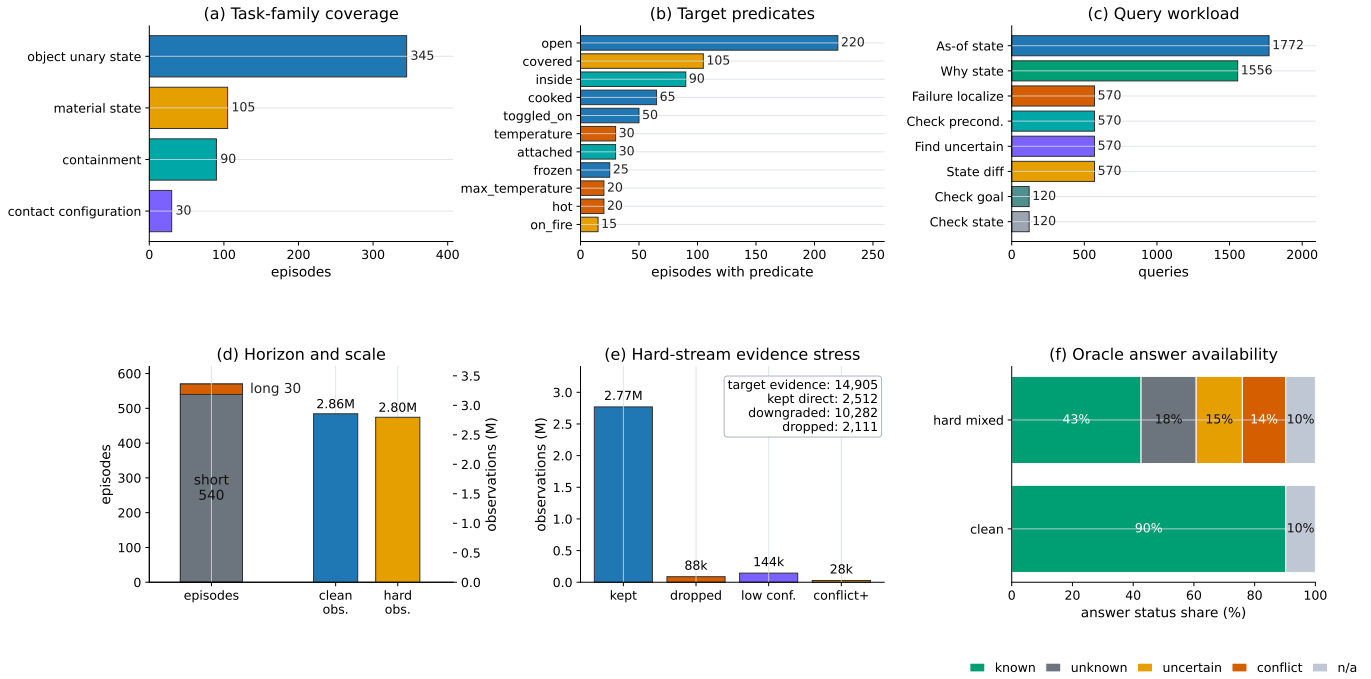


Fig. 1. Benchmark artifact profile. EviStateBench covers 570 simulator-grounded episodes over 36 activities, four task-state families, and 11 target predicates. The hard mixed stream stresses target evidence and shifts answer availability from mostly known clean answers to a mixture of known, unknown, uncertain, conflicting, and not-applicable answers.

be overstated. It means that explicit event-time and transaction-time maintenance is effective for these two query families. It does not mean that all of EviStateBench is easy, because the residual errors in Fig. 4 concentrate in set-valued uncertainty, state diffs, preconditions, failure localization, and goal views.

Fig. 5 explains where residual difficulty comes from. Retrieval fails broadly, while Temporal log and EviStateDB largely recover temporal and provenance answers. The remaining difficulty is therefore concentrated in answer shapes that require sets, diffs, preconditions, failure localization, and goal aggregation.

The interpretation should avoid two common mistakes. It should not treat EviStateDB as the oracle merely because it is the reference engine. It should also not claim that a lower score means a method is bad for all embodied AI tasks. A retrieval memory that performs poorly on state diffs may still be useful for open-ended question answering. The EAB claim is narrower and more useful: EviStateBench identifies which systems are good at temporal task-state view maintenance under a shared, reproducible workload.

VIII. LESSONS, LIMITATIONS, AND ARTIFACT AVAILABILITY

A. Lessons from the Benchmark Design

EviStateBench exposes five practical lessons. First, latest-state memory is not a sufficient abstraction because late evidence can revise as-of answers and contradict evidence can make a state uncertain or conflicting. Second, retrieval and view maintenance answer different questions: retrieval can

surface evidence, but task execution often needs a maintained predicate value, change set, or goal view. Third, provenance should be first-class because support and contradict evidence are needed for why queries, failure localization, and trust calibration. Fourth, goal views are not just convenience layers; they can fail even when individual predicates are partly correct. Fifth, validation reports are benchmark artifacts, since malformed query metadata or leaked answer fields can invalidate comparisons as much as model errors.

B. Limitations

EviStateBench makes a controlled isolation choice. The main artifact is simulator-grounded and structured; it does not claim to solve raw perception or low-level robot control. This is a limitation, but also a design requirement for a reproducible data-management benchmark, because folding perception and control into the main score too early would make failures hard to attribute. The reported baseline set is also intentionally conservative: neural memory, LLM-based agents, and full embodied pipelines fit the benchmark only when their public inputs, inference budgets, and output schemas can be made comparable.

C. Artifact Availability

The benchmark release provides public task specifications, observation streams, query sets, manifests, validation reports, build commands, baseline implementations, prediction schemas, and evaluator scripts. Hidden timelines and answer sets are used for evaluation. The minimal reproduction path

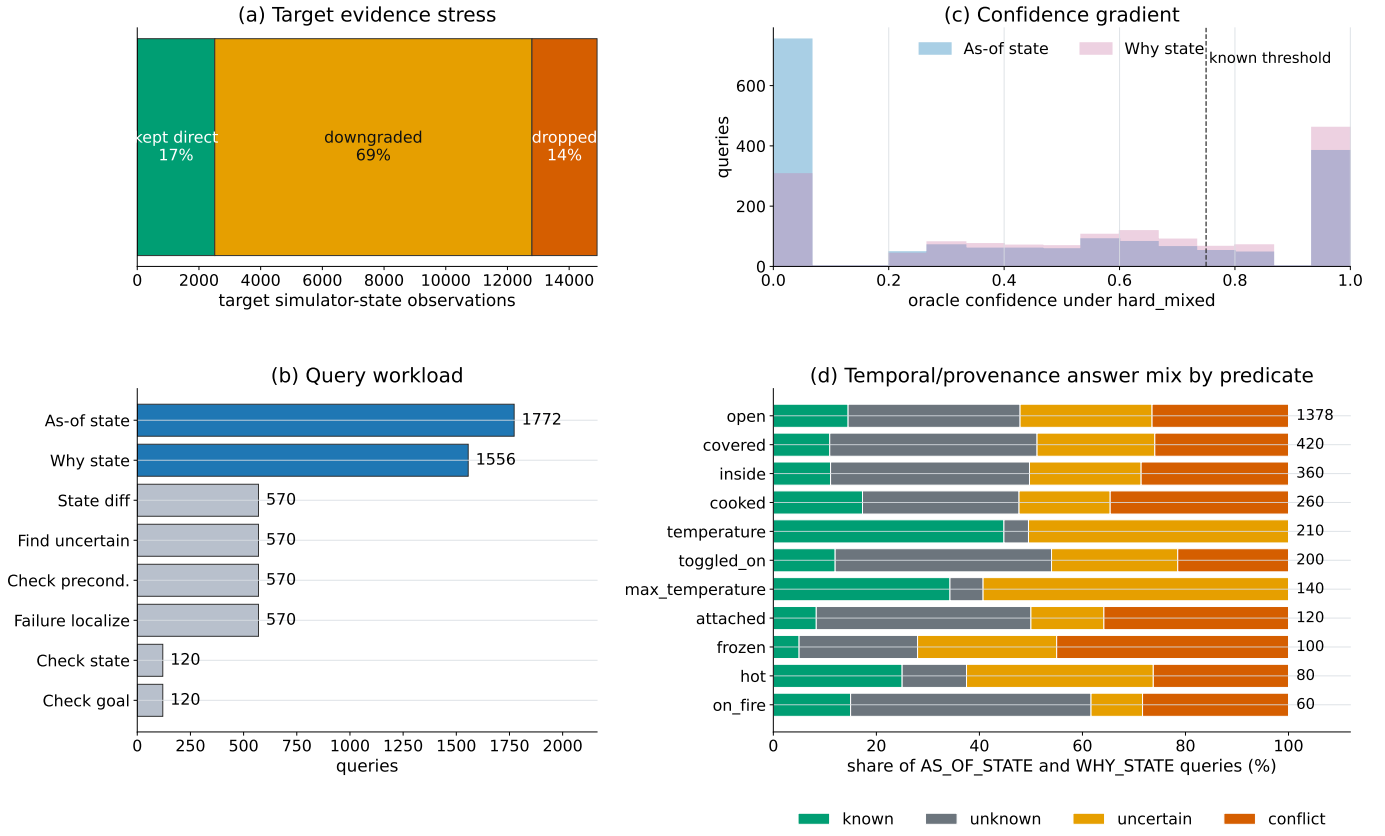


Fig. 2. Hard-split mechanism and query difficulty profile. The hard stream keeps only a minority of target simulator-state observations as direct evidence, while downgrading or dropping the rest. The resulting workload emphasizes temporal and provenance queries and produces graded known, unknown, uncertain, and conflicting answer conditions across target predicates.

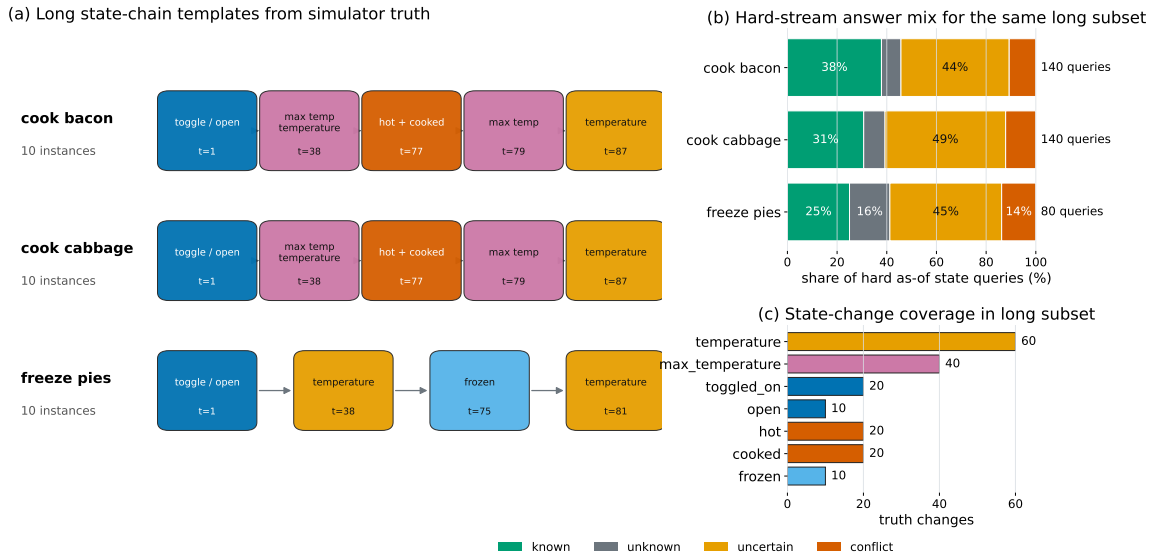


Fig. 3. Aggregated long state-chain templates. The 30-episode long subset adds multi-stage temporal maintenance cases, such as actuator, temperature, hot, cooked, and frozen state chains, to the larger short-transition benchmark.

builds or validates public artifacts, runs each baseline on the hard mixed stream, validates prediction schema, evaluates predictions, and regenerates the reported tables and figures.

IX. RELATED WORK

A. Embodied Task and Simulation Benchmarks

BEHAVIOR introduced a benchmark of everyday household activities with predicate-logic activity specifications and

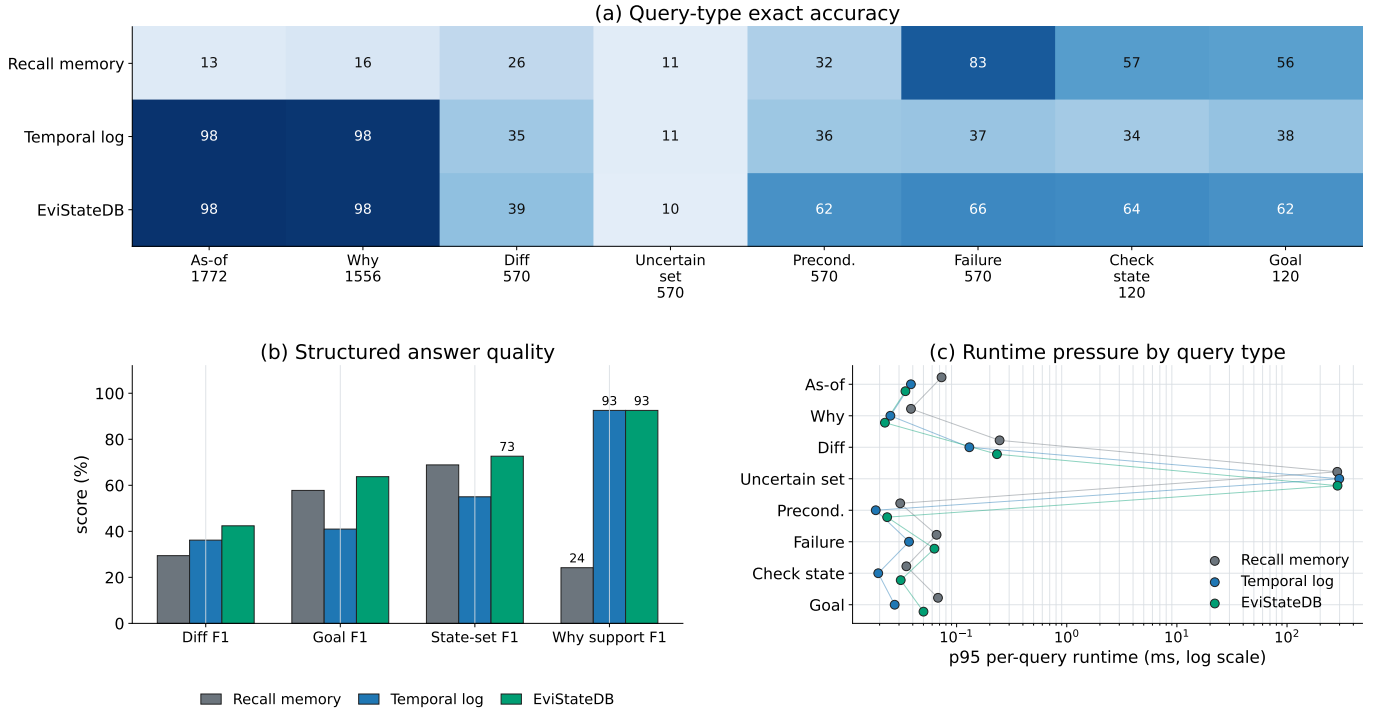


Fig. 4. Baseline results on the final hard split. The upper heatmap reports query-type exact accuracy, the lower-left panel reports structured answer metrics, and the lower-right panel reports p95 per-query runtime from prediction outputs. All methods cover the full 5,848-query workload.

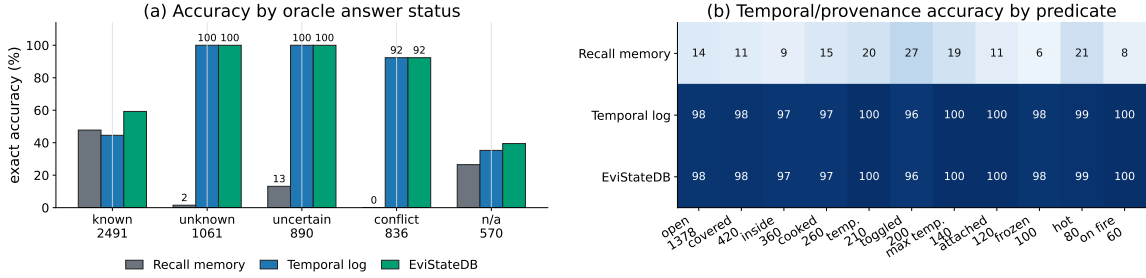


Fig. 5. Difficulty analysis by answer status and predicate. Recall memory degrades across answer statuses and target predicates, while explicit temporal-state baselines largely solve AS_OF_STATE and WHY_STATE.

simulation-based evaluation [1]. BEHAVIOR-1K scales this line of work to 1,000 activities and introduces OmniGibson as a simulation environment for realistic physics and rendering [2]. Habitat and Habitat 2.0 support embodied navigation and rearrangement evaluation [3], [4]; AI2-THOR provides an interactive 3D environment for visual AI [5]; VirtualHome represents household activities as executable programs [6]; and TEACH studies embodied agents that interact through dialogue [7]. EviStateBench uses the same general idea that task specifications and simulator truth can ground evaluation, but it changes the target from embodied task success to temporal task-state view maintenance.

ALFRED evaluates instruction following [9], and OpenEQA evaluates embodied question answering over episodic memory and active exploration [8]. These benchmarks ask whether an agent can act or answer language questions. EviStateBench

asks whether a system can maintain and query a temporal state view from imperfect evidence, without prescribing the system’s internal memory architecture.

B. Temporal, Stream, and Provenance Data Management

Temporal databases separate the time when a fact is valid in the modeled world from the time when a database records or knows the fact [10], [11]. Stream systems study continuous processing over arriving data [12], [25]. Incremental view maintenance studies how derived database state can be updated after base changes [13], [14]. Provenance and uncertain-data systems address evidence and noisy facts [15]–[17], [26]. EviStateBench brings these ideas into embodied task-state maintenance, where event-time and arrival-time mismatch, missing observations, conflicting evidence, and task-derived goal views appear together.

C. Experiment, Analysis, and Benchmark Papers

Database benchmark and EAB papers often contribute workload models, artifact rules, and diagnostic analyses rather than only a new algorithm. LDBC standardizes graph workloads [18], [19]; TSM-Bench benchmarks time-series systems [20]; and FinBench defines a financial graph transaction workload [21]. Recent EAB papers similarly contribute evaluation frameworks and fine-grained diagnostics, including SQLMorph [22], BEACON [23], and plan-based AQP analysis [24]. EviStateBench follows this pattern: the contribution is the benchmark artifact, workload, evaluator, and analysis of where baseline strategies succeed or fail.

X. CONCLUSION

EviStateBench is a data-management benchmark for temporal task-state view maintenance over embodied observation streams. It separates public observation streams and queries from hidden timelines and answers, evaluates systems through standardized QueryAnswers, and makes event time, arrival time, evidence, perturbations, and task-derived state views first-class benchmark concerns. EviStateDB provides a reference baseline engine, but the benchmark oracle comes from simulator truth and task specifications. The reported artifact and experiments show that temporal maintenance strategies outperform retrieval-style memory on this workload, while structured, set-valued, and task-derived queries remain open stress points for submitted systems.

AI-GENERATED CONTENT ACKNOWLEDGEMENT

The authors used AI-assisted tools during manuscript drafting, code editing, experiment-log summarization, and figure-preparation support. The authors reviewed, edited, and verified the submitted content and remain responsible for the correctness, originality, and integrity of the paper.

REFERENCES

- [1] S. Srivastava, C. Li, M. Lingelbach, R. Martin-Martin, F. Xia, K. Vainio, Z. Lian, C. Gokmen, S. Buch, C. K. Liu, S. Savarese, H. Gweon, J. Wu, and L. Fei-Fei, “BEHAVIOR: Benchmark for everyday household activities in virtual, interactive, and ecological environments,” in *Proceedings of the Conference on Robot Learning*, 2022.
- [2] C. Li *et al.*, “BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1,000 everyday activities and realistic simulation,” *arXiv preprint arXiv:2403.09227*, 2024.
- [3] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, “Habitat: A platform for embodied AI research,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9339–9347.
- [4] A. Szot, A. Clegg, E. Undersander, E. Wijmans, Y. Zhao, J. Turner, N. Maestre, M. Mukadam, D. S. Chaplot, O. Maksymets, A. Gokaslan, V. Vondrus, S. Dharur, F. Meier, W. Galuba, A. X. Chang, Z. Kira, V. Koltun, J. Malik, M. Savva, and D. Batra, “Habitat 2.0: Training home assistants to rearrange their habitat,” in *Advances in Neural Information Processing Systems*, 2021.
- [5] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, M. Deitke, K. Ehsani, D. Gordon, Y. Zhu, A. Gupta, and A. Farhadi, “AI2-THOR: An interactive 3d environment for visual AI,” *arXiv preprint arXiv:1712.05474*, 2017.
- [6] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba, “VirtualHome: Simulating household activities via programs,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8494–8502.
- [7] A. Padmakumar, J. Thomason, A. Shrivastava, P. Lange, A. Narayan-Chen, S. Gella, R. Piramuthu, G. Tur, and D. Hakkani-Tur, “TEACH: Task-driven embodied agents that chat,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.
- [8] A. Majumdar, A. Ajay, X. Zhang, P. Putta, S. Yenamandra, M. Henaff, S. Silwal, P. McVay, O. Maksymets, S. Arnaud, K. Yadav, Q. Li, B. Newman, M. Sharma, V. Berges, S. Zhang, P. Agrawal, Y. Bisk, D. Batra, M. Kalakrishnan, F. Meier, C. Paxton, A. Sax, and A. Rajeswaran, “OpenEQA: Embodied question answering in the era of foundation models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 16488–16498.
- [9] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox, “ALFRED: A benchmark for interpreting grounded instructions for everyday tasks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.
- [10] R. T. Snodgrass, *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.
- [11] C. S. Jensen and R. T. Snodgrass, “Temporal data management,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 11, no. 1, pp. 36–44, 1999.
- [12] B. Babcock, S. Babu, M. Datar, R. Motwani, and J. Widom, “Models and issues in data stream systems,” in *Proceedings of the ACM Symposium on Principles of Database Systems*, 2002, pp. 1–16.
- [13] A. Gupta and I. S. Mumick, “Maintenance of materialized views: Problems, techniques, and applications,” *IEEE Data Engineering Bulletin*, vol. 18, no. 2, pp. 3–18, 1995.
- [14] Y. Zhuge, H. Garcia-Molina, J. Hammer, and J. Widom, “View maintenance in a warehousing environment,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1995, pp. 316–327.
- [15] C. C. Aggarwal and P. S. Yu, “A survey of uncertain data algorithms and applications,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 5, pp. 609–623, 2009.
- [16] T. J. Green, G. Karvounarakis, and V. Tannen, “Provenance semirings,” in *Proceedings of the ACM Symposium on Principles of Database Systems*, 2007, pp. 31–40.
- [17] J. Cheney, L. Chiticariu, and W.-C. Tan, “Provenance in databases: Why, how, and where,” *Foundations and Trends in Databases*, vol. 1, no. 4, pp. 379–474, 2009.
- [18] O. Erling, A. Averbuch, J. Larriba-Pey, H. Chafi, A. Gubichev, A. Prat, M.-D. Pham, and P. Boncz, “The LDBC social network benchmark: Interactive workload,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 2022–2033, 2015.
- [19] A. Iosup, T. Hegeman, W. L. Ngai, S. Heldens, A. Prat-Pérez, T. Manhardt, H. Chafi, M. Capotă, N. Sundaram, M. Anderson, I. G. Tănase, Y. Xia, L. Nai, P. Boncz, and G. Siganos, “LDBC graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms,” *Proceedings of the VLDB Endowment*, vol. 9, no. 13, pp. 1317–1328, 2016.
- [20] A. Khelifati, M. Khayati, A. Dignös, D. Difallah, and P. Cudré-Mauroux, “Tsm-bench: benchmarking time series database systems for monitoring applications,” *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3363–3376, 2023.
- [21] S. Qi, B. Tong, J. Hu, H. Lin, Y. Pang, W. Yuan, S. Lyu, Z. Guo, K. Huang, X. Ba *et al.*, “The ldbc financial benchmark: Transaction workload,” *Proceedings of the VLDB Endowment*, vol. 18, no. 9, pp. 3007–3020, 2025.
- [22] M. Malekpour, M. Riahi, M. Lamothe, and A. Mhedhbi, “SQLMorph: Query mutation and fine-grained metrics for text-to-SQL evaluation,” in *Proceedings of the IEEE International Conference on Data Engineering*, 2026, experiment, Analysis, and Benchmark paper.
- [23] M. M. Najafi, X. Zhu, C. Kosyfaki, L. V. S. Lakshmanan, and R. Cheng, “BEACON: A benchmark for efficient and accurate counting of sub-graphs,” *arXiv preprint arXiv:2504.10948*, 2025.
- [24] P. Mu, A. Chaves Carniel, A. Barbalace, and A. Shaikhha, “[experiment, analysis, and benchmark] systematic evaluation of plan-based adaptive query processing,” *arXiv preprint arXiv:2511.16455*, 2025.
- [25] M. Garofalakis, J. Gehrke, and R. Rastogi, *Data Stream Management: Processing High-Speed Data Streams*. Springer, 2016.
- [26] N. N. Dalvi and D. Suciu, “Efficient query evaluation on probabilistic databases,” in *Proceedings of the International Conference on Very Large Data Bases*, 2007, pp. 864–875.